Subject Code: BMC204

Paper id: 252745

Roll No: | | | | | | | | | | | | |

MCA
(SEM II) THEORY EXAMINATION 2024-25
DATABASE MANAGEMENT SYSTEMS

TIME: 3 HRS **M.MARKS: 70**

Note: Attempt all Sections. In case of any missing data; choose suitably.

SECTION A

1. Attempt *all* questions in brief. 2 x 07 = 14

a) Discuss the term Data Independence.

Data Independence is the capacity to change the schema at one level of a database system without requiring changes to the schema at the next higher level. It is a central feature of the **Three-Schema Architecture** and is divided into two types:

- **Logical Data Independence:** The ability to change the conceptual schema (e.g., adding or deleting columns) without affecting the external schemas or application programs.
- **Physical Data Independence:** The ability to change the physical storage structures (e.g., switching from B-trees to Hashing) without changing the conceptual schema.

---

b) What do you mean by composite attributes?

Composite attributes are attributes that can be further divided into smaller sub-parts, which represent more basic attributes with independent meanings.

Example: An attribute **Name** can be decomposed into **First_Name**, **Middle_Name**, and **Last_Name**. Similarly, an **Address** can be broken down into Street, City, State, and Zip Code.

---

c) Define Primary Key with example.

A Primary Key is a specific choice of a minimal set of attributes (columns) that uniquely identifies each tuple (row) in a relation. It must satisfy two strict properties:

1. **Unique Value:** No two rows can have the same primary key value.
2. **Non-Null:** It cannot contain a NULL value.

Example: In a STUDENT table, Student_ID is a Primary Key because every student has a unique ID, whereas Name might not be unique.

---

d) What is the syntax of INSERT clause used in SQL?

The INSERT INTO statement is used to add new records to a table. There are two primary ways to write it:

Option 1: Specifying columns

SQL>INSERT INTO table_name (column1, column2, ...)VALUES (value1, value2,...);

Option 2: Without specifying columns (must match table order)

SQL>INSERT INTO table_nameVALUES (value1, value2, value3, ...);

---

e) Define the term Functional Dependency.

Functional Dependency (FD) is a constraint that describes the relationship between attributes in a relation. If R is a relation and X and Y are subsets of attributes in R, we say that X functionally determines Y (denoted as X→Y) if each X value is associated with precisely one Y value.

In simpler terms, if you know the value of X, you can determine the value of Y.

R(rno,name,dob,grade),FD{rno→name, rno→dob, rno→grade}

---

f) What do you mean by Transaction in databases?

A Transaction is a single logical unit of work that performs one or more database operations (like Read, Write, Delete). For a transaction to be considered successful, it must adhere to the **ACID** properties:

- **Atomicity:** All or nothing.
- **Consistency:** Moves the DB from one valid state to another.
- **Isolation:** Transactions don't interfere with each other.
- **Durability:** Changes are permanent once committed.

---

g) Define Time Stamping in terms of concurrency control.

Time Stamping is a non-locking concurrency control technique used to order transactions based on their start times. Each transaction is assigned a unique identifier called a **Timestamp (TS)**.

- The DBMS ensures that if TS(T1)<TS(T2), the effect on the database is as if T1 executed entirely before T2.
- It uses **Read_TS(X)** and **Write_TS(X)** on data items to detect conflicts and prevent non-serializable behavior by rolling back younger transactions that try to access data modified by older ones (or vice versa, depending on the protocol).

SECTION B

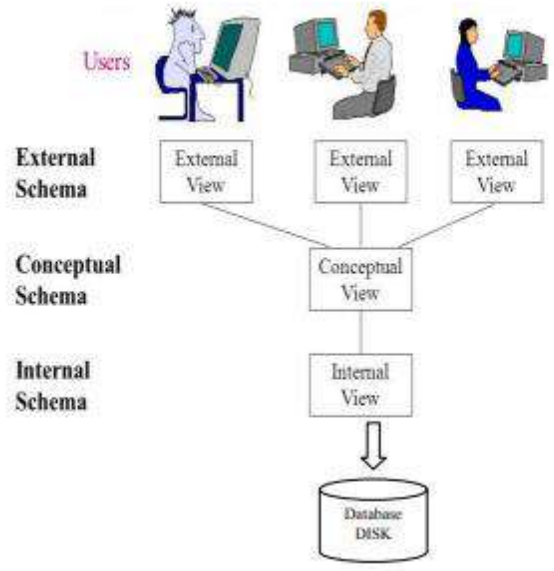
2. Attempt any three of the following. *(7 × 3 = 21)*

a) **What is three schema architecture? Discuss in detail**.

The **Three-Schema Architecture**, also known as the **ANSI-SPARC Architecture**, was designed to separate the user applications from the physical database. Its primary goal is to provide **Data Independence**, ensuring that changes at one level do not necessitate changes at others.

The Three Levels of Architecture

An early proposal for a standard terminology and general architecture database, a system was produced in 1971 by the DBTG (Data Base Task Group) appointed by the Conference on data Systems and Languages. The architecture is divided into three distinct levels of abstraction:



1. External Level (View Schema)

This is the highest level of abstraction and is closest to the users.

- **Definition:** It describes only the part of the database that is relevant to a specific user group.
- **Purpose:** It simplifies the user interaction by providing multiple "views" of the data while hiding irrelevant or sensitive details.
- **Example:** In a university database, a student sees their grades and tuition info, while a registrar sees student records and course enrollments.

2. Conceptual Level (Logical Schema)

This is the intermediate level that describes the structure of the **entire database** for a community of users.

- **Definition:** It hides the details of physical storage and concentrates on describing entities, data types, relationships, user operations, and constraints.
- **Purpose:** This is where the database administrator (DBA) defines the logical structure of the data.
- **Key Focus:** What data is stored and what relationships exist between those data points.

3. Internal Level (Physical Schema)

This is the lowest level of abstraction, closest to the actual storage.

- **Definition:** It describes the physical storage structure and access paths (e.g., indexes, hashing, and record ordering).
- **Purpose:** It deals with how the data is actually stored on disks or other storage devices.
- **Key Focus:** Efficiency, storage optimization, and data compression.

Without this separation, any change to the way data is stored on a hard drive would require rewriting every single application program that uses that data. By using the Three-Schema Architecture, we can upgrade your hardware or change our table structures with minimal disruption to the end-user experience.

---

b) Discuss basic operations of Relational Algebra with examples.

Relational algebra is the theoretical foundation of relational databases. It consists of a set of operations used to manipulate and query data stored in tables (relations).

1. Select Operation (σ): The **Select** operation identifies a subset of tuples (rows) from a relation that satisfy a specific condition (predicate). It acts as a horizontal filter.

Notation: σ p(R)

Where σ is used to represent SELECTION

R is used to represent RELATION

p is the logic formula

Example: Suppose we want the row(s) from STUDENT Relation where "AGE" is 20

$\sigma_{AGE=20}$ (STUDENT)

2. Project Operation (π): The **Project** operation selects specific columns (attributes) from a relation and discards the rest. It acts as a vertical filter. Notably, it automatically removes duplicate rows in the resulting set.

Notation : Π a(R)

Where Π is used to represent PROJECTION

R is used to represent RELATION

a is the attribute list

Example: Suppose we want the names of all students from STUDENT Relation.

Π NAME(STUDENT)

3. Union Operation (∪): The **Union** operation combines all tuples from two relations. For this to work, the relations must be **Union Compatible** (same number of attributes and identical data types for corresponding columns).

Notation: R ∪ S

Where R is the first relation S is the second relation

A union operation must hold the following condition:

o R and S must have the attribute of the same number.

o Attribute domains must be compatible.

o Duplicate tuples are eliminated automatically.

Example: we want all the names from STUDENT and EMPLOYEE relation.

Π NAME(STUDENT) ∪ Π NAME(EMPLOYEE)

4. Intersection Operation(∩): The **Intersection** operation provides common all tuples from two relations. Like Union, the relations must be Union Compatible.Suppose there are two tuples R and S.

Notation: R ∩ S
Π CUSTOMER_NAME (BORROW) ∩ Π CUSTOMER_NAME (DEPOSITOR)
5. Set Difference Operation (-): This operation finds tuples that are present in the first relation but **not** in the second. Like Union, the relations must be Union Compatible.
Notation: R - S
Ex: Finds all the tuples that are present in **Books** but not in **Articles**.
Π author (Books) − Π author (Articles)

6. Cartesian product Operation (X**):** The **Cartesian product** (or Cross Product) combines every tuple of the first relation with every tuple of the second relation. If Relation A has n rows and Relation B has m rows, the result has n * m rows.
Notation: E X D
σauthor = 'Smith'(Books X Articles)
Output − Yields a relation, which shows all the books and articles written by Smith.

7. Join Operations(⋈): Join Operation in DBMS are binary operations that allow us to combine two or more relations. A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by ⋈. Join is defined as a cartesian product followed by a selection operation.
(X, σ) -> ⋈

- Operation: (EMPLOYEE ⋈ SALARY)
- **Example:** If we have ComputerScience_Students and Electronics_Students, to get a list of all students in both departments:

8. Rename Operation (ρ): The RENAME operation is used to rename the output of a relation. It is denoted by **rho** (ρ).

Sometimes it is simple and suitable to break a complicated sequence of operations and rename it as a relation with different names.

ρ X (R)

R is the result of the sequence of operation or expression which is saved with the name X.

Example-1 rename the relation Student as Male Student and the attributes of Student – RollNo, SName as (Sno, Name).

- **ρ MaleStudent(Sno, Name) πRollNo, SName(Student)**

---

c) What do you mean by Normalization? Discuss 1NF, 2NF, and 3NF.

Normalization is the process of organizing data in a database to reduce **redundancy** (duplication) and improve **data integrity**. The primary goal is to isolate data so that additions, deletions, and modifications can be made in just one table and then propagated through the rest of the database using defined relationships.
By following normalization rules, we eliminate anomalies:

- **Insertion Anomaly:** Being unable to insert data because some other data is missing.
- **Update Anomaly:** Updating a value in one place but leaving old copies elsewhere.
- **Deletion Anomaly:** Losing unintentional data when deleting a specific record.

1. First Normal Form (1NF): For a table to be in 1NF, it must meet the following criteria:

- Each table cell must contain a **single (atomic) value**.
- There are no repeating groups or multi-valued attributes (e.g., you cannot have multiple phone numbers in one cell).
- Each record must be unique (usually identified by a Primary Key).

Example: If a table has a column Content containing multiple values, it violates 1NF. To fix it, we must split those into separate rows.

Course	Content
Programming	Java,Python
DBMS	Oracle,MySQL

Course	Content
Programming	Java
Programming	Python
DBMS	Oracle
DBMS	MySQL

2. Second Normal Form (2NF): To reach 2NF, a table must:

1. Already be in **1NF**.
2. Have **no partial functional dependencies**.

A partial dependency occurs when a non-prime attribute (a column not part of the primary key) depends on only *part* of a composite primary key. If your primary key is a single column, and the table is in 1NF, it is automatically in 2NF.

Example: In a Project_Assignment table with (Emp_ID, Proj_ID,Emp_Name,Hours) a composite key (Employee_ID, Project_ID), Emp_Name depends only on Emp_ID. This is a partial dependency. To fix it, move Emp_Name to a separate Employee table.

Emp_ID	Proj_ID	Emp_Name	Hours

Emp_ID	Emp_Name

Emp_ID	Proj_ID	Hours

3. Third Normal Form (3NF): To reach 3NF, a table must:

1. Already be in **2NF**.
2. Have **no transitive functional dependencies**.

A transitive dependency occurs when a non-prime attribute depends on another non-prime attribute, rather than directly on the primary key. In simpler terms: "A depends on B, and B depends on the Primary Key."

Example: In a Department table where Dept_ID is the primary key, if you have Dept_Name and Manager_id, and Manager_Phone depends on the Manager_id (not the Dept_ID), you have a transitive dependency. You should move the manager details to their own table.

Emp_ID	Emp_Name	City	Pincode

City	Pincode

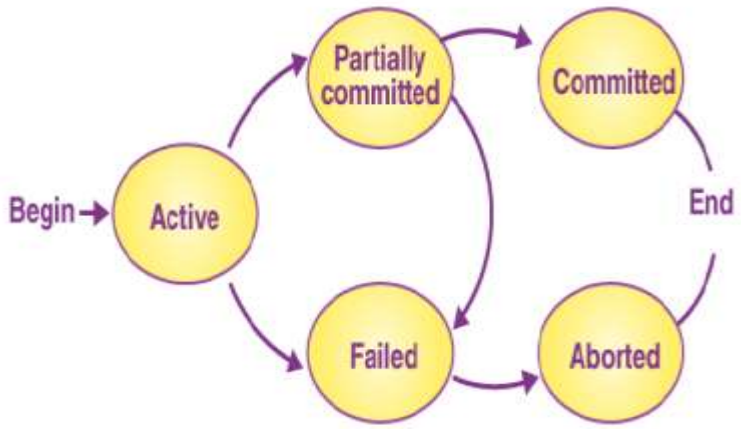
Emp_ID	Emp_Name	City

---

d) Discuss various states of a transaction.

A transaction in a Database Management System (DBMS) is a logical unit of processing that must be treated as a single, indivisible entity. During its execution, it transitions through several specific states. These states ensure the **ACID** properties (Atomicity, Consistency, Isolation, Durability) are maintained, particularly during system failures.

The Transaction State Diagram



A transaction typically moves through the following five states:

1. Active State

This is the initial state where the transaction begins.

- The transaction stays in this state while its operations (Read or Write) are being executed.
- If all operations execute successfully, it moves to the **Partially Committed** state.
- If a failure occurs (e.g., a logic error or system crash), it moves to the **Failed** state.

2. Partially Committed State

A transaction enters this state after the final statement has been executed.

- At this point, the changes are still residing in the main memory (buffer) and have not yet been physically saved to the disk.
- The system performs final checks (like consistency and serializability) before allowing the transaction to fully commit.

3. Committed State

The transaction enters this state after the system successfully completes all checks and ensures the data is permanently recorded on the disk.

- Once committed, the transaction is "final."
- The changes cannot be undone even if the system crashes later.
- This state fulfills the **Durability** property of ACID.

4. Failed State

A transaction enters the failed state if an error occurs during the **Active** or **Partially Committed** states.

- This could be due to hardware failure, software bugs, or a violation of database constraints.
- Once in this state, the transaction can no longer proceed.

5. Aborted State

After a transaction fails, the database must ensure that any partial changes made by the transaction are removed so the database returns to its previous consistent state.

- The process of undoing changes is called a **Rollback**.
- Once the rollback is complete, the transaction is in the **Aborted** state.
- From here, the system can either **Restart** the transaction (if the failure was transient) or **Kill** it (if the error was logical)

---

e) What are various problems encountered during concurrency control? Discuss each of them.

When multiple transactions are executed simultaneously in a multi-user system, they can interfere with one another if not properly managed. These interferences lead to **concurrency problems**, which compromise the integrity and consistency of the database.

The four primary problems encountered are:

1. Lost Update Problem: This occurs when two transactions, T_1 and T_2, read the same data and update it, but one update "overwrites" the other without including the previous change.

- **The Scenario:**
    1. T_1 reads a balance of 100.
    2. T_2 reads the same balance of 100.
    3. T_1 adds 50 and writes 150.
    4. T_2 adds 30 and writes 130.
- **The Result:** The 50 added by T_1 is completely lost. The final balance is 130 instead of 180.

2. Dirty Read Problem (Temporary Update): A dirty read occurs when a transaction reads data that has been updated by another transaction that has **not yet committed**. If that second transaction later fails and rolls back, the first transaction is left with "dirty" data that technically never existed.

- **The Scenario:**
    1. T_1 updates a value from 100 to 500.
    2. T_2 reads the value as 500.
    3. T_1 fails and performs a **Rollback**. The value reverts to 100.
- **The Result:** T_2 is working with the value 500, which is no longer valid in the database.

3. Unrepeatable Read Problem: This happens when a transaction reads the same data item twice but finds different values each time because another transaction modified and committed that data in between the reads.

- **The Scenario:**
  1. T_1 reads a value (e.g., Price = 50).
  2. T_2 updates that Price to 75 and **Commits**.
  3. T_1 reads the same Price again to perform a calculation and finds it is now 75.
- **The Result:** T_1 experiences inconsistency within its own execution flow.

4. Phantom Read Problem: The phantom problem is similar to an unrepeatable read but involves a **set of rows** rather than a single value. It occurs when a transaction executes a query to retrieve a set of rows, and a second transaction inserts or deletes rows that fit the search criteria before the first transaction finishes.

- **The Scenario:**
  1. T_1 queries all students with a grade > 80 (finds 5 students).
  2. T_2 inserts a new student with a grade of 90 and **Commits**.
  3. T_1 runs the same query again.
- **The Result:** T_1 now sees 6 students. The new record appeared like a "phantom."

SECTION C

Question 3

Attempt any one: *(7 marks)*

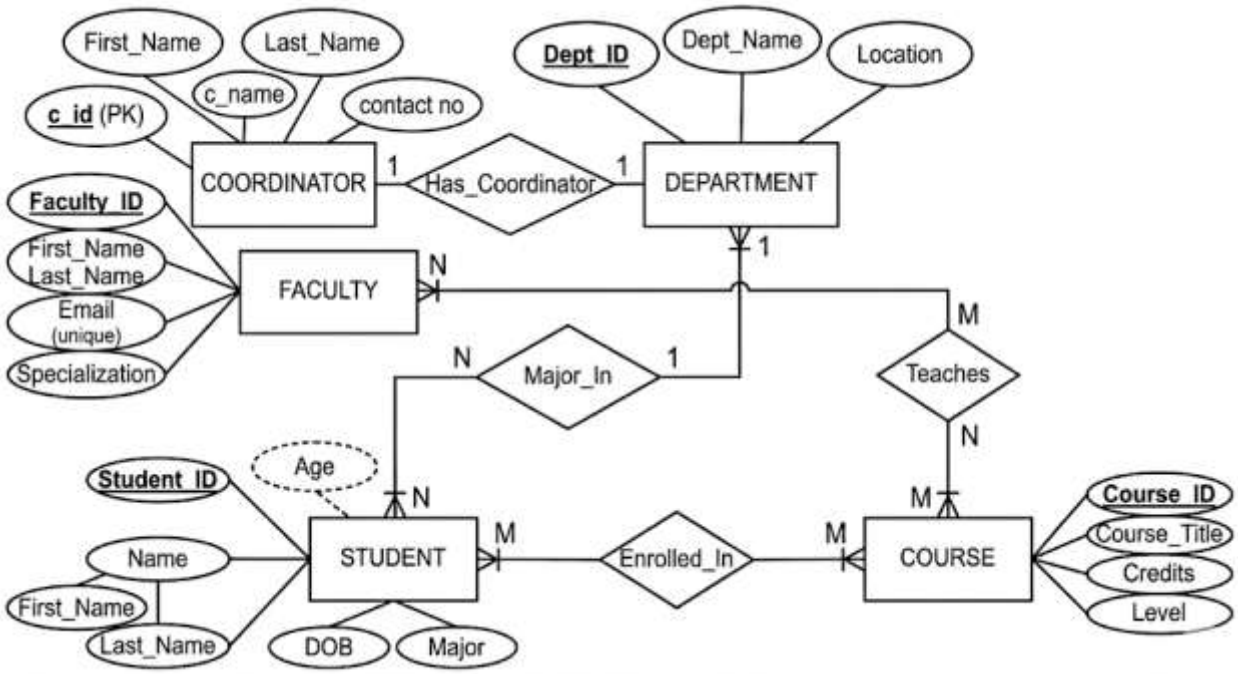
a) Discuss ER diagram with the help of an example.

An **Entity-Relationship (ER) Diagram** is a visual representation of the logical structure of a database. It describes how data is stored and how "entities" (objects or concepts) relate to one another.

Components of an ER Diagram

Component	Shape	Description
Entity	Rectangle	An object or concept (e.g., Student, Car, Course).
Attribute	Oval	Properties or traits of an entity (e.g., Name, Age).
Relationship	Diamond	How entities interact (e.g., Student *Enroll* in Course).
Primary Key	Underlined Text	An attribute that uniquely identifies an entity.

Example: University Management System



Cardinality (The "How Many" Rule)

- **One-to-One (1:1):** One cordinator manages one department.
- **One-to-Many (1:N):** One department has many students.
- **Many-to-Many (M:N):** Many students enroll in many different courses.

Uses of ER Diagrams

1. **Visual Communication:** It helps developers and stakeholders agree on the data structure before writing any SQL code.
2. **Database Design:** It serves as a blueprint for creating the physical tables. For example, a "Relationship" in an ER diagram often becomes a "Foreign Key" in a real table.
3. **Redundancy Check:** It helps identify where data might be duplicated unnecessarily (linking back to the normalization concepts we discussed).

---

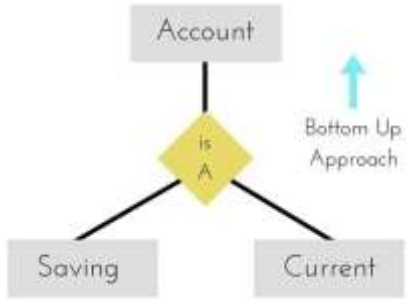
b) Explain the terms Generalization, Specialization and Aggregation.

In advanced ER modeling, these concepts (often called Enhanced Entity-Relationship or EER features) are used to handle complex relationships and data hierarchies.

Generalization

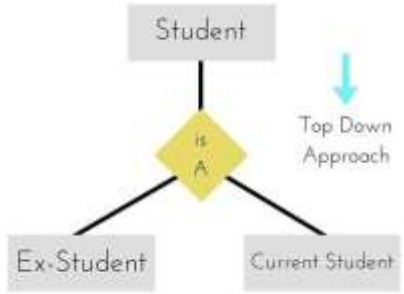
Generalization is the **bottom-up** process. It is the exact opposite of specialization. It combines multiple entity sets that share common features into a single high-level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity. It's more like Superclass and Subclass system. Entities are combined to form a more generalised entity, in other words, sub-classes are combined to form a super-class.

For example, Saving and Current account types entities can be generalised and an entity with name Account , which covers both.



Specialization: Specialization is a **top-down** process where a higher-level entity (Superclass) is divided into lower-level entities (Subclasses) based on distinguishing characteristics.

For Ex: Consider the entity **Student**. They might be current student or pass out students. While both shares attributes like Name and ID, only the Ex-Student has attributes like package while the current student has semester. It is Represented by a triangle labeled **"ISA"** (is-a).



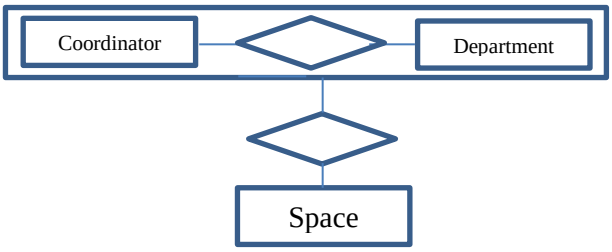
"An entity is a specialized version of another."

Aggregation

Aggregation is an abstraction used when a relationship between two entities needs to be treated as a single entity so it can participate in another relationship.

For Example: Suppose a **Coordinator** *manages* a **Department**.An **Office_Space** is *assigned to* that specific management pairing.we can't easily link **Office_Space** to the diamond "Manages." Instead, we

wrap the Coordinator-Manages-Department group in a box (Aggregation) and link the Office_Space to that box. It allows for relationships between relationships.



Question 4

Attempt any one: *(7 marks)*

a) What are the various Integrity Constraints? Discuss each of them.

Integrity constraints are a set of rules used to maintain the quality and consistency of information within a relational database. They ensure that data insertion, updating, and other processes are performed in a way that data integrity is not compromised.

In a Database Management System (DBMS), these constraints are generally categorized into four main types:

1. Entity Integrity Constraint: This rule ensures that every table has a unique identity and that each row can be specifically identified.

- **The Rule:** A primary key column cannot contain **NULL** values.
- **Purpose:** Since a primary key is used to identify individual records, a null value would make it impossible to distinguish or reference that specific row.
- **Syntax:** create table student(rno number(3) primary key);

2. Domain Integrity Constraint: Domain integrity ensures that all data items in a column fall within a defined set of valid values.

- **The Rule:** Values must adhere to the defined data type, format, and range for that specific column.
- **Examples:** * An "Age" column might be restricted to integers between 0 and 120.
    - A "Gender" column might only allow 'M', 'F', or 'O'.
- **Purpose:** It prevents invalid data types (like entering text into a currency field) from entering the system.
- **Syntax:** create table student(gender char(1),check (gender in('M', 'F', 'O')));

3. Referential Integrity Constraint: This constraint maintains the relationship between two tables. It involves Foreign Keys.

- **The Rule:** A value in a Foreign Key column must either match a value in the Primary Key of the referenced table or be NULL.
- **Mechanism:** If Table A references Table B, you cannot delete a record in Table B if that record is still being pointed to by Table A.
- **Purpose:** It prevents "orphaned" records, ensuring that relationships between data entities remain intact and valid.
- create table fine(fine_id number(3)primary key,rno number(3) references student);

4. Key Integrity Constraint: Key constraints ensure that specific attributes (or sets of attributes) uniquely identify an entity within an entity set.

- **The Rule:** In any given relation, there can be multiple "Candidate Keys," but one must be chosen as the Primary Key. All key values must be unique.
- **Purpose:** It ensures that no two rows in a table are exact duplicates of each other, maintaining the distinctness of data entries.
- create table student(rno number(3) primary key ,mobile_no number(10) unique);

---

b) Discuss PL/SQL Procedures and Functions with example.

In PL/SQL, **Procedures** and **Functions** are subprograms—blocks of code that performs specific tasks and can be reused.

PL/SQL Procedures: A procedure is a subprogram used to perform a specific action. It does not necessarily return a value; it is just like procedures in other programming languages. The procedure contains a header and a body.

Header: The header contains the name of the procedure and the parameters or variables passed to the procedure.

Body: The body contains a declaration section, execution section and exception section similar to a general PL/SQL block.

- Syntax Focus: Defined using CREATE OR REPLACE PROCEDURE.
- Execution: Called using the EXECUTE command or within another PL/SQL block.

When you want to create a procedure or function, you have to define parameters .There is three ways to pass parameters in procedure:

1. IN parameters: The IN parameter can be referenced by the procedure or function. The value of the parameter cannot be overwritten by the procedure or the function.
2. OUT parameters: The OUT parameter cannot be referenced by the procedure or function, but the value of the parameter can be overwritten by the procedure or function.
3. INOUT parameters: The INOUT parameter can be referenced by the procedure or function and the value of the parameter can be overwritten by the procedure or function. A procedure may or may not return any value.

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[ (parameter [,parameter]) ]
IS
[declaration_section]
BEGIN
executable_section
[EXCEPTION
exception_section]
END [procedure_name];
```

Ex: write the procedure code to add bonus in marks in student table.

```
CREATE OR REPLACE PROCEDURE Add_Bonus (
s_id IN NUMBER,
bonus IN NUMBER
) IS
BEGIN
   UPDATE Student  SET Marks = Marks + bonus WHERE ID =s_id;
   DBMS_OUTPUT.PUT_LINE('Marks updated successfully.');
END;
/
```

Call a procedure

```
BEGIN
   Add_Bonus(2,500);
END;
```

2. PL/SQL Functions: A function is designed to calculate and return a single value. Unlike a procedure, a function *must* contain a RETURN statement.

- **Syntax Focus:** Defined using CREATE OR REPLACE FUNCTION.
- **Execution:** Can be used in SQL statements (like SELECT) or assigned to a variable.

Example: Calculating Percentage

This function takes the total marks and returns the percentage.

```
CREATE OR REPLACE FUNCTION Get_Percentage (
   p_marks IN NUMBER,
```

```
   p_total IN NUMBER
) RETURN NUMBER IS
   v_percent NUMBER;
BEGIN
   v_percent := (p_marks / p_total) * 100;
   RETURN v_percent;
END;
/
```

Call a Function

```
DECLARE
   result NUMBER;
BEGIN
   result := Get_Percentage(85, 100);
   DBMS_OUTPUT.PUT_LINE(result);
END;
```

Question 5

Attempt any one: *(7 marks)*

a) Define Minimal Cover for a set of functional dependencies. Discuss with the help of example.

A **Minimal Cover** (also known as a Canonical Cover) of a set of functional dependencies (FDs) is the most simplified version of that set. It removes all redundancies while ensuring that the original logic and constraints of the data remain exactly the same.

Three Conditions for a Minimal Cover

For a set of functional dependencies F to be a minimal cover, it must satisfy these three rules:

1. **Right-Hand Side Singularity:** Every FD must have only a single attribute on its right side. (e.g., A →BC must be split into A→ B and A→ C).
2. **No Extraneous Attributes:** You cannot remove any attribute from the left side of an FD without changing the closure (the set of implied dependencies).
3. **No Redundant Dependencies:** You cannot remove an entire FD from the set without losing information.

Example : Let's find the Minimal Cover for the following set of FDs:

F = {A →BC, ABC→ E, EF →G}

Step 1: Singleton Right-Hand Side

Ensure every dependency points to only one attribute. In this case, A→BC, we would split it.

- A→B, A→C

Step 2: Remove Extraneous Attributes

Look at FDs with multiple attributes on the left.

Let's examine ABC→E.

- Calculate the closure of A. Since A→B,A→C ,ABC→E the A^+=ABCE, so A alone can determine E.

Let's examine EF→G.

But E^+=E and G^+=G so both are needed to determine G.

New Set: {A →B, A →E, EF→ G, A→ C}

Step 3: Remove Redundant Dependencies

Now we check if any full dependency can be deleted.

Let's see if A→B is necessary.

- If we remove A→B, can we still get from A to B using the other rules?
- Find the closure of A by using this FD A^+=ABCE
- Find the closure of A without using this FD A^+=ACE
- There is no way to reach B from A. So, A→B **stays**.

Let's see if A→E is necessary.

- If we remove A→E, can we still get from A to E using the other rules?

- Find the closure of A by using this FD A^+=ABCE
- Find the closure of A without using this FD A^+=ABC
- There is no way to reach E from A. So, A→E **stays**.

Let's see if EF→G is necessary.

- If we remove EF→G, can we still get from EF to G using the other rules?
- Find the closure of EF by using this FD EF^+=EFG
- Find the closure of EF without using this FD EF^+=EF
- There is no way to reach B from A. So, EF→G **stays**.

Let's see if A→C is necessary.

- If we remove A→C, can we still get from A to C using the other rules?
- Find the closure of A by using this FD A^+=ABCE
- Find the closure of A without using this FD A^+=ABE
- There is no way to reach B from A. So, A→C **stays**.

Minimal Cover:
{A →B, A →E, EF→ G, A→ C}
{A →BCE, EF→ G}

---

b) Discuss lossless join with the help of an example.
In Database Management Systems, a lossless join decomposition ensures that when a relation is split into two or more tables, we can join them back without losing or generating extra (spurious) data. After decomposition, the original table can be perfectly reconstructed using a natural join.
Condition for Lossless Join
A decomposition of relation R into R_1 and R_2 is lossless if at least one of the following is true:

- R1∩R2→R1
  or
- R1∩R2→R2

That means: The common attributes should functionally determine at least one table.
Example of Lossless Join
A Relation R(A,B,C) having Functional Dependency :{A→B} Decompose into:R1(A,B) and R2(A,C)
Step 1: Find Common Attribute
Common attribute = A
Step 2: Check Condition
We have:

- A→B

So:

- A determines all attributes of R1(A,B)
- Condition satisfied → Lossless Join

Step 3: Verify with Table
Original Table R

A	B	C
1	X	P
2	Y	Q

After Decomposition

R_1(A, B)

A	B
1	X
2	Y

R_2(A, C)

A	C
1	P
2	Q

Step 4: Natural Join -Joining R_1 and R_2 on A:

A B C

1 X P

2 Y Q

We get the original table back exactly So this is a lossless join decomposition.

Question 6

Attempt any one: *(7 marks)*

a) What do you mean by ACID properties of a transaction? Discuss each of them.

In Database Management Systems, a **transaction** is a set of operations that are executed as a single logical unit of work. To ensure reliability and correctness of database operations, every transaction must satisfy the **ACID properties**.

ACID stands for **Atomicity, Consistency, Isolation, and Durability**.

1. Atomicity: Atomicity means "all or nothing"**. A transaction is considered atomic if either:**

- All operations are executed successfully, or
- None of them are executed (in case of failure)

If any part of the transaction fails, the entire transaction is **rolled back**.

Example : In a bank transfer:

- □1000 is debited from Account A
- □1000 is credited to Account B

If the credit operation fails, the debit is also cancelled.

Thus, no partial execution is allowed.

2. Consistency: Consistency ensures that the database remains in a valid state **before and after the transaction. A transaction must follow all Integrity constraints.**

Example: If a database rule states that balance cannot be negative, then after any transaction, this condition must still hold. It guarantees that only valid data is stored in the database.

3. Isolation: Isolation means that multiple transactions executing simultaneously should not interfere with each other**. Each transaction works as if it is the** only transaction in the system**.**

Example: If two users are updating the same account at the same time, One user should not see incomplete changes made by the other. This prevents problems like dirty reads, non-repeatable reads, etc.

4. Durability: Durability ensures that once a transaction is **committed**, its changes are **permanently stored** in the database. Even in case of System crash or Power failure the committed data will not be lost.

Example: After successful completion of a bank transaction, the updated balance will remain saved permanently.

ACID properties are essential for maintaining:

- Data integrity
- Reliability
- Consistency of the database

---

b) **Write short notes on Testing of Serializability.**

In Database Management Systems, **serializability** is a concept used to ensure that the execution of concurrent transactions produces the **same result as some serial (one-by-one) execution**.Testing of serializability is important to check whether a given **schedule** is correct and free from inconsistency.

Types of Serializability

1. Conflict Serializability
2. View Serializability

1. Conflict Serializability Test (Using Precedence Graph)

Steps:

1. Create a **precedence (serialization) graph**

2. Each transaction is represented as a **node**
3. Draw a **directed edge** from Ti to Tj if:
    - An operation of Ti conflicts with an operation of Tj .
    - And Ti's operation occurs before Tj.

Two operations inside a schedule are called conflicting if they meet these three conditions:

1. They belong to two different transactions.
2. They are working on the same data piece.
3. One of them is performing the WRITE operation.

Decision Rule:

- If the graph has **no cycle** → Schedule is **conflict serializable**
- If the graph has a **cycle** → Not serializable

2. View Serializability Test: A schedule is view serializable if it is **view equivalent** to a serial schedule.

Conditions:

1. **Initial Read**: Initial read must be on the same piece of data. If transaction t1 is reading "A" from database in schedule S1, then in schedule S2, t1 must read A.

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

2. **Final write**: A final write must be the same between both the schedules.Example: If a transaction t1 updated A at last in S1, then in S2, t1 should perform final write as well.

T1	T2	T3
Write(A)		
	Read(A)	
		Write(A)

Schedule S1

T1	T2	T3
	Read(A)	
Write(A)		
		Write(A)

Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

3. **Update Read:** If in a schedule S, a transaction Ti executes read(Q), and that value was produced by transaction Tj (if any), then in schedule S' also transaction Ti must read the value of Q that was produced by the same write(Q) operation of transaction Tj. For example, if T3 reads the value of B updated by T2 in schedule S then in S', T3 must read the value of B updated by T2.

Updated Read

T1	T2	T3
	W(B)	
		R(B)
R(A)		

S

T1	T2	T3
R(A)		
	W(B)	
		R(B)

S'

Question 7

Attempt any one: *(7 marks)*

a) Discuss Locking Techniques for concurrency control.

In Database Management Systems, **locking techniques** are used to control concurrent access to data items so that **data consistency and isolation** are maintained when multiple transactions execute simultaneously.

A **lock** is a mechanism that restricts access to a data item.

1. **Lock-Based Protocols**: One way to ensure isolation is to require that data items be accessed in a mutually exclusive manner; that is, while one transaction is accessing a data item, no other transaction can modify that data item. The most common method used to implement this requirement is to allow a transaction to access a data item only if it is currently holding a lock on that item.

Locks: There are various modes in which a data item may be locked.

1. **Shared**. If a transaction T_i has obtained a **shared-mode lock** (denoted by S) on item Q, then T_i can read, but cannot write, Q.
2. **Exclusive**. If a transaction T_i has obtained an **exclusive-mode lock** (denoted by X) on item Q, then T_i can both read and write Q.

	S	X
S	true	false
X	false	false

Lock-compatibility matrix

It is required that every transaction **request** a lock in an appropriate mode on data item Q, depending on the types of operations that it will perform on Q. The transaction makes the request to the concurrency-control manager. The transaction can proceed with the operation only after the concurrency-control manager **grants** the lock to the transaction. The use of these two lock modes allows multiple transactions to read a data item but limits write access to just one transaction at a time.

To access a data item, transaction T_i must first lock that item. If the data item is already locked by another transaction in an incompatible mode, the concurrency- control manager will not grant the lock until all incompatible locks held by other transactions have been released. Thus, T_i is made to wait until all incompatible locks held by other transactions have been released.

T_1: lock-X(B); read(B);
B := B-50;
write(B);
unlock(B);
lock-X(A);
read(A);
A := A + 50;
write(A);
unlock(A).

2. The Two-Phase Locking Protocol: One protocol that ensures serializability is the two-phase locking protocol. This protocol requires that each transaction issue lock and unlock requests in two phases:

Growing phase. A transaction may obtain locks, but may not release any lock.

Shrinking phase. A transaction may release locks, but may not obtain any new locks.

Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests.

T2:*lock-X(B);*
read(B);
B := B-50;

write(B);
lock-X(A);
read(A);
A := A + 50;
write(A);
unlock(B);
unlock(A).

On the other hand, transactions T_1 is not two phase.

3. **Strict two-phase locking protocol:** Schedules should be cascadeless. Cascading rollback may occur under two-phase locking. Consider the partial schedule below. Each transaction observes the two-phase locking protocol, but the failure of T_5 after the read(A) step of T_7 leads to cascading rollback of T_6 and T_7.

T_5	T_6	T_7
lock-X(A)		
read(A)		
write(A)		
unlock(A)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
		lock-S(A)
		read(A)

Cascading rollbacks can be avoided by a modification of two-phase locking called the **strict two-phase locking protocol**. This protocol requires not only that locking be two phase, *but also that all exclusive-mode locks taken by a transaction be held until that transaction commits.* This requirement ensures that any data written by an uncommitted transaction are locked in exclusive mode until the transaction commits, preventing any other transaction from reading the data.

T_5	T_6	T_7
lock-X(A)		
read(A)		
write(A)		
commit;		
unlock(A)		
	lock-X(A)	
	read(A)	
	write(A);	
	commit;	
	unlock(A)	
		lock-S(A)
		read(A)

4. Rigorous two-phase locking protocol: Another variant of two-phase locking is the rigorous two-phase locking protocol, which requires *that all locks be held until the transaction commits.*

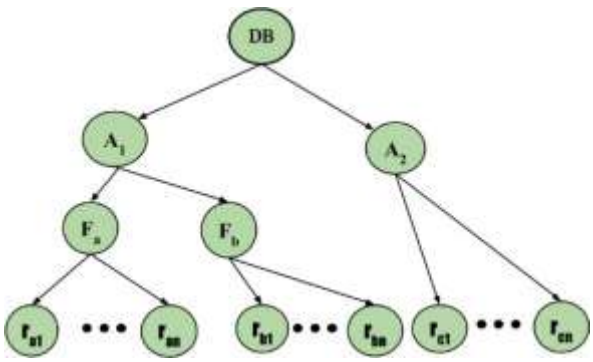
---

b) What do you mean by Granularity? Discuss Multiple Granularity.

A drawback of Concurrency Control schemes is if a transaction T_i needs to access the entire database and a locking protocol is used, then T_i must lock each item in the database. Suppose another transaction just needs to access a few data items from a database, so locking the entire database seems to be unnecessary.

Granularity – It is the size of the data item allowed to lock. Now *Multiple Granularity* means hierarchically breaking up the database into blocks that can be locked and can be tracked . Such a hierarchy can be represented graphically as a tree.
For example, consider the tree, which consists of four levels of nodes. The highest level represents the entire database. Below it is nodes of type area; the database consists of exactly these areas. The area has children nodes which are called files. Every area has those files that are its child nodes. No file can span more than one area.
Finally, each file has child nodes called records. As before, the file consists of exactly those records that are its child nodes, and no record can be present in more than one file. Hence, the levels starting from the top level are:

- database
- area
- file
- record



When a transaction locks a node, in either shared or exclusive mode, the transaction also implicitly locks all the descendants of that node in the same lock mode. For example, if transaction T_i gets an explicit lock on file F_c in exclusive mode, then it has an implicit lock in exclusive mode on all the records belonging to that file. It does not need to lock the individual records of F_c explicitly.
A more efficient way to attain this is to introduce a new lock mode, called *Intention lock mode*.
Intention Mode Lock –In addition to S and X lock modes, there are three additional lock modes with multiple granularities:

- **Intention-Shared (IS):** explicit locking at a lower level of the tree but only with shared locks.
- **Intention-Exclusive (IX):** explicit locking at a lower level with exclusive or shared locks.
- **Shared & Intention-Exclusive (SIX):** the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive mode locks.

The compatibility matrix for these lock modes are described below:

	IS	IX	S	SIX	X
IS	✔	✔	✔	✔	✗
IX	✔	✔	✗	✗	✗
S	✔	✗	✔	✗	✗
SIX	✔	✗	✗	✗	✗
X	✗	✗	✗	✗	✗

IS : Intention Shared
IX : Intention Exclusive
S : Shared
X : Exclusive
SIX : Shared & Intention Exclusive

The multiple-granularity locking protocol uses the intention lock modes to ensure serializability. It requires that a transaction T_i that attempts to lock a node must follow these protocols:

1. Transaction T_i must follow the lock-compatibility matrix.
2. Transaction T_i must lock the root of the tree first, and it can lock it in any mode.

3. Transaction T_i can lock a node in S or IS mode only if T_i currently has the parent of the node-locked in either IX or IS mode.
4. Transaction T_i can lock a node in X, SIX, or IX mode only if T_i currently has the parent of the node-locked in either IX or SIX modes.
5. Transaction T_i can lock a node only if T_i has not previously unlocked any node (i.e., T_i is two-phase).
6. Transaction T_i can unlock a node only if T_i currently has none of the children of The multiple-granularity protocol requires that locks be acquired in top-down (root-to-leaf) order, whereas locks must be released in bottom-up (leaf to-root) order.